

VUNA Football League

Management System - Project Documentation

Next.js 14 / Zustand / Supabase / Tailwind CSS

VUNA Football League — Management System

- > Version: 1.0.0
- > Framework: Next.js 14.1.0 (App Router)
- > State Management: Zustand 4.5 with persist middleware
- > Database: Supabase (PostgreSQL)
- > Auth: Supabase Auth (cookie-based SSR sessions)
- > Styling: Tailwind CSS
- > Deployment: Vercel

Table of Contents

1. [Project Overview](#1-project-overview)
2. [Architecture & Data Flow](#2-architecture--data-flow)
3. [Authentication System](#3-authentication-system)
4. [Route Map](#4-route-map)
5. [Page-by-Page Documentation](#5-page-by-page-documentation)
 - 5.1 Login (`/auth/login`)
 - 5.2 Dashboard (`/`)
 - 5.3 Live Scores (`/live`)
 - 5.4 Fixtures (`/fixtures`)
 - 5.5 Standings (`/standings`)
 - 5.6 Teams (`/teams`)
 - 5.7 Players (`/players`)
 - 5.8 Admin Panel (`/admin`)
 - 5.9 Reports (`/reports`)
 - 5.10 Setup (`/auth/setup`)
6. [Component Catalog](#6-component-catalog)
7. [API Reference](#7-api-reference)
8. [Database Schema](#8-database-schema)
9. [State Management](#9-state-management)
10. [Key Libraries & Utilities](#10-key-libraries--utilities)

1. Project Overview

The VUNA Football League Management System is a full-featured web application for managing football league operations. It supports three user roles with different access levels:

```

Role | Login Method | Scope
---|---|---

```

****Super Admin**** | Email + Password (Supabase Auth) | Full CRUD — manage teams, players, fixtures, standings, accounts

****Team Account**** | Username + Password (auto-generated email `team\_teamname-001@vfl.local`) | View own team's data, generate player credentials

****Player Account**** | Username + Password (auto-generated email `player\_username@vfl.local`) | View personal dashboard, stats, match history, leaderboards

Key UI Features:

- Global search across teams, players, and fixtures
- AM/PM time input component
- Public live scores page (no login required)
- Team logo upload and display
- Player performance charts and achievement badges
- Fixture generation with round-robin algorithm
- Lineup builder with formation visualization
- Team of the Round computation
- Data import/export (JSON/CSV/PDF/PNG)

2. Architecture & Data Flow

[illegible]

% Browser (Client)
% Zustand Store (vfl-app-state)
% Teams Players Fixtures !• persisted
% Auth Session !• NOT persisted
% Admin/Team Player Dashboard Public Pages
% (api/team/data) (api/player/data) (api/public/
% HTTP Fetch
% Next.js API Routes
% Auth APIs Data APIs Sync/Management AP
% (login, (team, (sync, upload, player, fixtures, players
% logout, player, fixtures, players
% session) admin) teams, transfers)
% Supabase Client
% Supabase
% PostgreSQL Database
% Tables: teams, players, fixtures, match_events,
% admin_users, team_accounts, player_profiles,
% credential_generation_logs, notifications, transf
% team_lineups
% Auth (Supabase Auth)
% Users: admins (email/pwd), team_accounts, players
% Storage (Team Logos)

Data Flow by Role

Admin Flow:

1. Admin manages data locally in Zustand (persisted to localStorage)
2. Manually syncs to Supabase via "Sync to DB" buttons (`/api/sync/\*`)
3. Fetches from Supabase on login via `/api/admin/data`

Team Account Flow:

1. On login, fetches data from Supabase via `/api/team/data`
2. Receives own team's players, all fixtures, all teams for standings
3. Teams/Reports tabs hidden — cannot manage these entities

Player Account Flow:

1. On login, fetches data from Supabase via `/api/player/data`
2. Receives own profile, team info, team fixtures, all fixtures, match events, all players, standings
3. Read-only — no edit/delete/add buttons visible
4. Dashboard shows personal stats, match history, performance chart, badges, leaderboards

3. Authentication System

Three Auth Modes

Mode | DB Table | Auth Provider | Login Route | Session Key

|---|---|---|---|---

Admin | `admin\_users` | Supabase Auth (email/pwd) | `POST /api/auth/admin-login` | Supabase session cookie

Team Account | `team\_accounts` | Supabase Auth (auto-generated email) | `POST /api/auth/team-login` | Supabase session cookie

Player | `player\_profiles` | Supabase Auth (auto-generated email) | `POST /api/auth/player-login` | Supabase session cookie

Authentication Flow

1. User submits credentials via `<LoginForm>`
2. Zustand login action POSTs to appropriate endpoint
3. Endpoint validates credentials against Supabase Auth
4. On success, session cookie is set (HTTP-only, server-managed)
5. Zustand store updates: `isAdmin`, `userProfile`, `currentTeamAccount`
6. `<AppShell>` detects authentication change and renders the full app layout
7. On subsequent requests, middleware validates the session and enforces role-based access

Middleware Security (`middleware.ts`)

The middleware runs on every server request and enforces:

- Rate limiting** — Separate limits for auth endpoints (10 req/min), public API (120 req/min), and other API (80 req/min)
- Session validation** — Checks Supabase session on every request
- Role-based page access**:
 - Players may only visit: `/`, `/fixtures`, `/standings`, `/players`
 - Admins may visit all pages, including `/admin` and `/admin/*`
 - Team accounts have no middleware restriction (limited by tab visibility instead)
- API route protection** — Blocks unauthorized access to non-public API routes
- Redirect logic** — Players accessing restricted pages get redirected to `/fixtures`
- Security headers** — X-Content-Type-Options, X-Frame-Options, Referrer-Policy, Permissions-Policy, HSTS (production)

Credential Generation

Player accounts are created via the credential generation system:

- Username format**: `PLAYERNAME_TEAMNAME_001`
- Password format**: `PLAYERNAME_001`
- Email format**: `player_username@vfl.local`
- Implementation**: `lib/player-credentials.ts::generatePlayerCredentialsForScope()`
- Uses Supabase Admin API to create auth users with auto-confirmed emails
- Creates corresponding row in `player_profiles` table

4. Route Map

Pages

Route | Page Component | Visibility | Tab in Nav

---|---|---|---

`/` | DashboardPage | Admin, Team, Player | Dashboard
`/live` | LiveFeed (on page) | **Public** | Live
`/fixtures` | FixtureList | All authenticated | Fixtures
`/standings` | StandingsTable | All authenticated | Standings
`/teams` | TeamForm | **Admin** (hidden from Team/Player) | Teams
`/players` | PlayerList + PlayerStats + TeamOfRound + LineupBuilder | **Admin** (hidden from Team/Player) | Players
`/admin` | AdminPanel | **Admin only** (adminOnly tab) | Admin
`/admin/transfers` | Transfers page | Admin | Link from Admin
`/reports` | ReportView | **Admin** (hidden from Team/Player) | Reports
`/auth/login` | LoginForm | Unauthenticated only | —
`/auth/setup` | First-time admin setup | Unauthenticated only | —

API Routes

Route | Methods | Auth | Purpose

---|---|---|---

`/api/auth/admin-login` | POST | Public | Admin email/password login
`/api/auth/admin-signup` | POST | Public | Admin registration
`/api/auth/team-login` | POST | Public | Team account username login
`/api/auth/player-login` | POST | Public | Player username login
`/api/auth/player-register` | POST | Public | Player self-registration
`/api/auth/logout` | POST | Authenticated | Clear session
`/api/auth/session` | GET | Public | Returns session info or `{authenticated: false}`
`/api/admin/data` | GET | Admin | Fetch all teams, players, fixtures
`/api/admin/create-team-account` | POST | Admin | Create a new team account
`/api/admin/team-accounts` | GET | Admin | List all team accounts
`/api/admin/generate-player-credentials` | POST | Admin | Generate player login credentials

```

`/api/team/data` | GET | Team Account | Fetch team-scoped data
`/api/team/generate-player-credentials` | POST | Team Account | Generate credentials for own team
`/api/player/data` | GET | Player | Fetch player dashboard data
`/api/fixtures` | GET | Authenticated | List all fixtures
`/api/fixtures/:id` | GET/PUT | Authenticated | Single fixture read/update
`/api/fixtures/generate` | POST | Admin | Generate round-robin fixtures
`/api/teams` | GET/POST | Authenticated (admin for POST) | List/create teams
`/api/teams/:id` | GET/PUT/DELETE | Authenticated | Single team CRUD
`/api/teams/:id/lineups` | GET/POST | Authenticated | List/create lineups
`/api/teams/:id/lineups/:lineupId` | GET/PUT/DELETE | Authenticated | Single lineup CRUD
`/api/players` | GET/POST | Authenticated (admin for POST) | List/create players
`/api/players/:id` | GET/PUT/DELETE | Authenticated | Single player CRUD
`/api/notifications` | GET/PATCH | Authenticated | List/mark-read notifications
`/api/public/live` | GET | **Public** | Live scores feed
`/api/transfers` | GET | Admin | Transfer history
`/api/sync/teams` | POST | Admin | Sync teams to Supabase
`/api/sync/players` | POST | Admin | Sync players to Supabase
`/api/sync/fixtures` | POST | Admin | Sync fixtures to Supabase
`/api/upload/logo` | POST | Authenticated | Upload team logo
`/api/upload/logo/check` | GET | Authenticated | Check storage bucket

```

5. Page-by-Page Documentation

5.1 Login (`/auth/login`)

Renders: <LoginForm>

Visibility: Unauthenticated users only

The login form has three modes switchable via pill tabs:

- ****Team mode:**** Username input + Password input + Sign In button
- ****Player mode:**** Username input (auto-uppercased) + Password input + Sign In as Player button
- ****Admin mode:**** Email input + Password input + Sign In as Admin button

On successful login, the user is redirected to /. Error messages display in a red alert box.

The login form is also rendered inline by <AppShell> when a user accesses any protected page without authentication.

5.2 Dashboard (`/`)

Visibility: Admin, Team Account, Player Account

Admin/Team Account View

1. ****Heading**** — "Season Control" (admin) or "Team Dashboard" + team name (team account)
2. ****Generate Fixtures button**** (admin) / ****Refresh button**** (team account)
3. ****Team welcome card**** (team only) — Team logo, name, rating
4. ****GeneratePlayerCredentials**** (team only) — Card to generate login credentials for team players
5. ****<MetricCards>**** — 4-stat grid:
 - Admin: Teams count, Goals, Matches, Players
 - Team: Team Players, Team Goals, Matches Played, Wins
6. ****<LeagueStats>**** (admin only) — Goals Scored, Goals Per Match, Biggest Win, Highest Round
7. ****Two-column layout:****
 - ****<UpcomingMatches>**** — Next round's matches with status badges
 - ****<TopFiveStandings>**** — Top 5 teams by points with logos, GD, rating

Player View

1. ****Heading**** — "Player Dashboard" + player name
2. ****<PlayerHero>**** — Profile card: photo (or placeholder), jersey number badge, name, captain badge, position, team name, rating badge, team logo
3. ****<PlayerStatsSummary>**** — Single card with 6 stat groups:
 - ****Attacking:**** Goals, Assists, Own Goals
 - ****Discipline:**** Yellow Cards, Red Cards
 - ****Goalkeeping:**** Saves, Penalty Saves, Clean Sheets, Goals Conceded
 - ****Defensive:**** Tackles, Interceptions, Blocks, Aerial Duels
 - ****Performance:**** Avg Rating, MOTM, Match Wins, Bonus 5 Saves
 - ****Miscellaneous:**** Errors Leading to Goal, Penalties Conceded

4. **Two-column layout**
 - `<PlayerFormGuide>` — Last 5 match results as W/L circles with rating color dots
 - **Upcoming Matches** card — Next 3 scheduled matches
5. `<PlayerPerformanceChart>` — SVG line chart of match ratings across rounds (0-10 scale)
6. `<PlayerBadges>` — Achievement badges:
 - First Goal (1+), Goal Scorer (5+), Sharp Shooter (10+), Playmaker (1+), Creator (5+), Star Player (1+ MOTM), Superstar (5+), Wall (1+ clean sheet), Safe Hands (5+ saves)
7. `<PlayerLeaderboards>` — Three top-5 tables: Top Scorers, Most Assists, Highest Rating (with rank medals, team logo, name, value, highlights current player)
8. `<PlayerMatchHistory>` — Expandable completed matches: score, W/L badge, rating badge, events. Expanded view shows date/time/venue and player-specific events

5.3 Live Scores (`/live`)

Visibility: Public (no login required)

Renders: `<LiveFeed>`

Data source: GET `/api/public/live` (uses anon key Supabase client)

- Auto-refreshes every 30 seconds
- Shows live/in-progress matches first, then today's upcoming matches
- Team crests displayed alongside team names
- No auth wrapper — `<AppShell>` bypasses login for `/live`

5.4 Fixtures (`/fixtures`)

Visibility: All authenticated users

Renders: `<FixtureList>`

1. **Header** — 4 summary stat cards: Rounds count, Total Matches, Live/In-progress, Completed
2. `<FixtureFilters>` (admin only) — Round/Team/Status dropdown filters + Export menu (JSON/PNG/PDF)
3. `<FixtureCreator>` (admin only) — Expandable form to add a fixture:
 - Home team dropdown, Away team dropdown
 - Round number, Date input, Time (via `<TimeInput>` component with 24h/12h toggle), Venue
4. **List of `<FixtureRoundPanel>` sections**, each containing:
 - "Round N" header with bye indicator (if a team has a bye)
 - `<FixtureCard>` items: home/away names, scores (if played), status badge, date/time/venue
 - Admin can drag-and-drop to reorder matches

Fixtures are sorted by date then time within each round. Empty dates sort last.

5.5 Standings (`/standings`)

Visibility: All authenticated users

Renders: `<StandingsTable>`

Column | Content

|---|---|

| Rank number

Team | Team logo (20×20px circular) + Team name

Rtg | Team rating

P | Matches played

W | Wins

D | Draws

L | Losses

GF | Goals for

GA | Goals against

GD | Goal difference

Pts | Points

Form | Last 5 results (W/D/L colored indicators)

Features:

- Download menu: JSON / PDF / PNG export

5.6 Teams (`/teams`)

Visibility: Admin only (tab hidden from Team/Player accounts)

Renders: `<TeamForm>` + grid of `<TeamCard>` items

1. **Add Team form** — Team name input, Add button, validation
2. **Action buttons** — Reset names, Reset all

3. **Grid of TeamCards**, each showing:

- Logo upload (via `<ImageUpload>`)
- Editable team name
- Player count badge
- Editable rating (admin, inline edit)
- Delete button (admin)
- "View Team Details" modal:
 - Overview stats: Rating, Player count, Y/R cards, Goals, Assists, Saves, Clean sheets, Avg player rating
 - Mini pitch formation preview (GK / DEF / MID / ATT)

5.7 Players (`/players`)

Visibility: Admin only (tab hidden from Team/Player accounts)

Four sub-tabs:

Tab 1: Players List

- Add Player button, CSV Import, Download CSV, Delete All
- Team/position filter dropdowns
- Grid of `<PlayerCard>`: name, number, position, team badge, rating, position-specific stats (GK: Saves/PS/CS/GC; DEF: T/INT/BLK/AD/G/A; MID: G/A/T/INT; ATT: G/A), captain crown icon, edit/delete buttons
- `<PlayerModal>` for add/edit: name, team, position, jersey number, captain checkbox

Tab 2: Statistics

- 4 metric cards: Players count, Avg rating, Total goals, Teams count
- Leading performer section
- 7 stat tables: Top Scorers, Top Assists, Yellow Cards, Red Cards, Saves (GK only), Tackles, Clean Sheets
- Download report button

Tab 3: Team of the Round

- Round selector dropdown
- Formation selector (4-3-3, 4-4-2, 3-5-2)
- Computes best 11 based on match events with scoring algorithm
- Displays in formation rows (GK / DEF / MID / ATT)
- Download as image

Tab 4: Lineup Builder

- **Lineup details:** Team selector, Name, Formation, Active checkbox, Save/Delete
- **Saved lineups:** Load/save from API
- **Formation builder:** Per-slot player assignments filtered by position
- **Preview panel:** Mini pitch visualization
- Supported formations: 4-3-3, 4-4-2, 3-5-2

5.8 Admin Panel (`/admin`)

Visibility: Admin only (adminOnly tab, middleware blocks non-admin)

Six sub-tabs:

Tab 1: Fixtures & Scores

- Generate Fixtures button + Repair Fixtures button
- Status notices for generation/repair results
- `<MatchEditor>` — Expandable list of all matches with:
 - Home/Away score inputs
 - Team selectors
 - Status selector (scheduled/in-progress/completed)
 - Date, Time (via `<TimeInput>`), Venue
 - Status badges (Live, Completed)
 - Issue flags
 - Embedded `<EventLog>`

Tab 2: Teams

- Same as `/teams` page: `<TeamForm>` + `<TeamCard>` grid

Tab 3: Players

- Add player form
- `<GeneratePlayerCredentials>`
- Team filter dropdown
- List of all players with edit/delete

Tab 4: Team Accounts

- List of existing team accounts (username, team, role)
- Create form: Team selector, Password, Show/hide toggle
- Displays generated credentials after creation

Tab 5: Database

- Sync Teams to Database button with status
- Sync Players to Database button with status
- Sync Fixtures to Database button with status

Tab 6: Import Data

- **4-step wizard:**
 1. **Upload** — File dropzone or paste JSON
 2. **Preview** — Shows teams/matches/players counts + team name mapping
 3. **Confirm** — Destructive warning, final confirm
 4. **Done** — Summary + auto-sync to database
- Handles ID remapping during import

5.9 Reports (`/reports`)

Visibility: Admin only (tab hidden from Team/Player)

Renders: `<ReportView>`

- Printable text report combining fixture schedule + standings
- Plain text format suitable for printing or copying

5.10 Setup (`/auth/setup`)

Visibility: Unauthenticated users (first time setup)

- First-time admin account creation form
- Creates the initial super admin in `admin_users` table
- Redirects to `/auth/login` after success (2-second delay)

6. Component Catalog

Layout Components (`components/layout/`)

Component | File | Purpose

|---|---|---|

AppShell | `app-shell.tsx` | Root wrapper — handles auth gating, renders spinner/login form/layout

AppHeader | `app-header.tsx` | Top bar: logo, title, search button, admin tools (export/import/reset), user info, notifications, logout

TabNav | `tab-nav.tsx` | Horizontal role-filtered tab bar: Dashboard, Live, Fixtures, Standings, Teams, Players, Admin, Reports

LoginForm | `login-form.tsx` | 3-mode login form (Team/Player/Admin) with mode tabs

Dashboard Components (`components/dashboard/`)

Component | File | Purpose | Visibility

|---|---|---|---|

MetricCards | `metric-cards.tsx` | 4-stat summary grid | All authenticated

LeagueStats | `league-stats.tsx` | 4 league-wide stats | Admin only

UpcomingMatches | `upcoming-matches.tsx` | Next round matches list | All authenticated

TopFiveStandings | `top-five-standings.tsx` | Top 5 teams table | All authenticated

Fixture Components (`components/fixtures/`)

Component | File | Purpose

|---|---|---|

FixtureList | `fixture-list.tsx` | Full fixtures page with filters, creator, round panels

****FixtureRoundPanel**** | `fixture-round.tsx` | Single round section with matches
****FixtureCard**** | `fixture-card.tsx` | Individual match display card
****FixtureFilters**** | `fixture-filters.tsx` | Round/Team/Status filters + Export
****FixtureCreator**** | `fixture-creator.tsx` | Add fixture form

Admin Components (`components/admin/`)

Component | File | Purpose

|---|---|---|

****AdminPanel**** | `admin-panel.tsx` | 6-tab admin panel (Fixtures, Teams, Players, Accounts, Database, Import)
****MatchEditor**** | `match-editor.tsx` | Full match editing with scores, status, date/time, embedded EventLog
****EventLog**** | `event-log.tsx` | Match events CRUD (17 event types)
****DataImporter**** | `data-importer.tsx` | 4-step JSON import wizard

Player Components (`components/player/`)

Component | File | Purpose

|---|---|---|

****PlayerDashboard**** | `player-dashboard.tsx` | Player dashboard orchestrator
****PlayerHero**** | `player-hero.tsx` | Profile card with photo, jersey, name, rating, team
****PlayerStatsSummary**** | `player-stats-summary.tsx` | Comprehensive single-card stats with 6 groups
****PlayerFormGuide**** | `player-form-guide.tsx` | Last 5 match W/L circles
****PlayerPerformanceChart**** | `player-performance-chart.tsx` | SVG rating line chart
****PlayerMatchHistory**** | `player-match-history.tsx` | Expandable match list
****PlayerBadges**** | `player-badges.tsx` | Achievement badges with progress
****PlayerLeaderboards**** | `player-leaderboards.tsx` | Top 5 scorers/assists/rating tables

Standings Components (`components/standings/`)

Component | File | Purpose

|---|---|---|

****StandingsTable**** | `standings-table.tsx` | Full league table with form guide

Team Components (`components/teams/`)

Component | File | Purpose

|---|---|---|

****TeamForm**** | `team-form.tsx` | Team management form + card grid
****TeamCard**** | `team-card.tsx` | Individual team display with details modal

Player Management Components (`components/players/`)

Component | File | Purpose

|---|---|---|

****PlayerList**** | `player-list.tsx` | Player grid with filters, add, CRUD
****PlayerCard**** | `player-card.tsx` | Individual player display card
****PlayerModal**** | `player-modal.tsx` | Add/edit player form modal
****PlayerStats**** | `player-stats.tsx` | Statistics tables tab
****TeamOfRound**** | `team-of-round.tsx` | Best 11 formation display
****LineupBuilder**** | `lineup-builder.tsx` | Full lineup creation tool
****GeneratePlayerCredentials**** | `generate-player-credentials.tsx` | Credential generation card

Shared Components (`components/shared/`)

Component | File | Purpose

|---|---|---|

****TimeInput**** | `time-input.tsx` | Time input with 24h/12h toggle
****SearchModal**** | `search/search-modal.tsx` | Global search overlay
****Notifications**** | `notifications/notifications.tsx` | Notification bell with dropdown

7. API Reference

Authentication Endpoints

`POST /api/auth/admin-login`

• ****Body:**** `{ email: string, password: string }`

- **Response:** `{ user: { id, email }, session }` or `{ error: string }`
- **Rate limit:** 10 requests/minute

`POST /api/auth/team-login`

- **Body:** `{ username: string, password: string }`
- **Response:** `{ user: { id, teamId, displayName, username }, session }` or `{ error: string }`

`POST /api/auth/player-login`

- **Body:** `{ username: string, password: string }`
- **Response:** `{ user: { id, playerId, displayName, username }, session }` or `{ error: string }`

`GET /api/auth/session`

- **Returns:** `{ authenticated: false }` or `{ authenticated: true, role: "super\_admin"|"team\_account"|"player", profile: {...} }`
- Always returns 200 (no 401 for unauthenticated users)

Data Endpoints

`GET /api/admin/data`

- **Auth:** Admin session required
- **Returns:** `{ teams: Team[], players: Player[], fixtures: FixtureRound[] }`

`GET /api/team/data`

- **Auth:** Team account session required
- **Returns:** `{ teams: Team[], players: Player[], fixtures: FixtureRound[] }` (scoped to managed team)

`GET /api/player/data`

- **Auth:** Player session required
- **Returns:** `{ player, team, teamFixtures, allFixtures, matchEvents, allPlayers, standings }`

`GET /api/public/live`

- **Auth:** None (public)
- **Returns:** Live/in-progress and today's upcoming matches

Sync Endpoints

`POST /api/sync/teams`

- **Auth:** Admin
- **Body:** `{ teams: Team[] }`
- **Returns:** `{ idMap: Record<number, number> }` (old ID != new DB ID mapping)

`POST /api/sync/players`

- **Auth:** Admin
- **Body:** `{ players: Player[], teamIdMap: Record<number, number> }`
- **Returns:** Operation status

`POST /api/sync/fixtures`

- **Auth:** Admin
- **Body:** `{ fixtures: FixtureRound[], teamIdMap: Record<number, number> }`
- **Returns:** Operation status

Upload Endpoint

`POST /api/upload/logo`

- **Auth:** Authenticated
- **Body:** FormData with `file` (image) and `teamId`
- **Validates:** File type (JPEG/PNG/WebP), size (< 2MB), dimensions
- **Returns:** `{ url: string }` — public URL of uploaded logo

8. Database Schema

Tables

`teams`

Column | Type | Description

|---|---|---|

id | BIGSERIAL PK | Auto-incrementing ID
name | TEXT NOT NULL | Team name
logo | TEXT | Logo URL (Supabase Storage)
rating | NUMERIC DEFAULT 3.0 | Team rating
created_at | TIMESTAMPTZ | Creation timestamp

`players`

Column | Type | Description

|---|---|---|

id | BIGSERIAL PK | Auto-incrementing ID
team_id | BIGINT FK !' teams(id) | Team membership
name | TEXT NOT NULL | Player name
position | TEXT | GK / DEF / MID / ATT
number | INTEGER | Jersey number
goals, assists, own_goals | INTEGER | Attacking stats
yellow_cards, red_cards | INTEGER | Discipline stats
saves, penalty_saves | INTEGER | Goalkeeper stats
clean_sheets, goals_conceded | INTEGER | Defensive stats
tackles, interceptions, blocks | INTEGER | Defensive actions
aerial_duels_won | INTEGER | Aerial stats
motm, match_wins | INTEGER | Performance stats
bonus_5_saves | INTEGER | Bonus stat
errors_leading_to_goal | INTEGER | Negative stat
penalties_conceded | INTEGER | Negative stat
captain | BOOLEAN DEFAULT false | Captain flag
rating | NUMERIC DEFAULT 5.0 | Average rating
match_ratings | JSONB | Per-match ratings `{ "matchId": rating }`
created_at, updated_at | TIMESTAMPTZ | Timestamps

`fixtures`

Column | Type | Description

|---|---|---|

id | BIGSERIAL PK | Auto-incrementing ID
round | INTEGER | Round number
home_id | BIGINT FK !' teams(id) | Home team
away_id | BIGINT FK !' teams(id) | Away team
home_score, away_score | INTEGER | Match scores (null until played)
status | TEXT | scheduled / in-progress / completed / live
date | TEXT | Match date (DD/MM/YYYY or YYYY-MM-DD)
time | TEXT | Match time (HH:MM or H:MM AM/PM)
venue | TEXT | Venue name
events | JSONB | Match events array

`match\_events`

Column | Type | Description

|---|---|---|

id | BIGSERIAL PK | Auto-incrementing ID
fixture_id | BIGINT FK !' fixtures(id) | Associated fixture
player_id | BIGINT FK !' players(id) | Player involved
team_id | BIGINT FK !' teams(id) | Player's team
event_type | TEXT | Event type
minute | INTEGER | Minute of event
created_at | TIMESTAMPTZ | Creation timestamp

`admin\_users`

Column | Type | Description

|---|---|---|

id | UUID PK !' auth.users(id) | Supabase Auth user ID
email | TEXT | Admin email
created_at | TIMESTAMPTZ | Creation timestamp

`team\_accounts`

Column	Type	Description
id	UUID PK	auth.users(id) Supabase Auth user ID
team_id	BIGINT FK	teams(id) Managed team
username	TEXT UNIQUE	Login username
display_name	TEXT	Display name
role	TEXT	coach / captain
created_at	TIMESTAMPTZ	Creation timestamp

`player\_profiles`

Column	Type	Description
id	UUID PK	auth.users(id) Supabase Auth user ID
player_id	BIGINT FK	players(id) Associated player
username	TEXT (unique index)	Login username
must_change_password	BOOLEAN DEFAULT true	Force password change
display_name	TEXT	Player display name
jersey_number	INTEGER	Jersey number override
position	TEXT	Position override
photo_url	TEXT	Profile photo URL
bio	TEXT	Player biography
created_at	TIMESTAMPTZ	Creation timestamp

`credential\_generation\_logs`

Column	Type	Description
id	BIGSERIAL PK	Auto-incrementing ID
generated_by	UUID FK	auth.users(id) Admin/team who generated
team_id	BIGINT FK	teams(id) Associated team
scope	TEXT	admin / team
players_affected	INTEGER	Count of generated/regenerated
created_at	TIMESTAMPTZ	Creation timestamp

`notifications`

Column	Type	Description
id	BIGSERIAL PK	Auto-incrementing ID
user_id	UUID	Target user
type	TEXT	Notification type
title	TEXT	Notification title
message	TEXT	Notification body
read	BOOLEAN DEFAULT false	Read status
created_at	TIMESTAMPTZ	Creation timestamp

`team\_lineups`

Column	Type	Description
id	BIGSERIAL PK	Auto-incrementing ID
team_id	BIGINT FK	teams(id) Associated team
name	TEXT	Lineup name
formation	TEXT	4-3-3 / 4-4-2 / 3-5-2
slots	JSONB	Player slot assignments
is_active	BOOLEAN	Active lineup flag
created_at, updated_at	TIMESTAMPTZ	Timestamps

`transfers`

Column	Type	Description
id	BIGSERIAL PK	Auto-incrementing ID
player_id	BIGINT FK	players(id) Player transferred
from_team_id	BIGINT FK	teams(id) Source team
to_team_id	BIGINT FK	teams(id) Destination team
transfer_date	TIMESTAMPTZ	Transfer date
created_at	TIMESTAMPTZ	Creation timestamp

9. State Management

Zustand Store (`lib/store/index.ts`)

The app uses Zustand with persist middleware for state management. The store is divided into four slices:

```
export type AppStore = AuthSlice & TeamsSlice & FixturesSlice & PlayersSlice;
```

Persisted data (localStorage key: vfl-app-state):

- `teams: Team[]`
- `fixtures: FixtureRound[]`
- `players: Player[]`

Non-persisted data (re-initialized on each session):

- Auth state (`currentTeamAccount`, `isAdmin`, `userProfile`, `authLoading`)
- `teamDataLoaded` flag

Store Slices

Auth Slice (`lib/store/auth-slice.ts`)

State | Type

|---|---|

`currentTeamAccount` | `TeamAccount` | null

`isAdmin` | `boolean`

`userProfile` | `UserProfile` | null

`authLoading` | `boolean`

`teamDataLoaded` | `boolean`

Action | Description

|---|---|

`initializeAuth()` | Fetches `/api/auth/session` and populates state

`loginAdmin(email, password)` | POST to `/api/auth/admin-login`

`loginTeamAccount(username, password)` | POST to `/api/auth/team-login`

`loginPlayer(username, password)` | POST to `/api/auth/player-login`

`logout()` | POST to `/api/auth/logout`, resets all state

`setTeamDataLoaded(v)` | Sets loading flag

Teams Slice (`lib/store/teams-slice.ts`)

Action | Description

|---|---|

`setTeams(teams)` | Bulk replace teams

`addTeam(team)` | Add single team

`deleteTeam(id)` | Remove by ID

`updateTeam(id, data)` | Partial update

`resetTeams()` | Reset to defaults

`setTeamLogo(id, logo)` | Set logo URL

`teamName(id)` | Get team name by ID

`getTeam(id)` | Get team object by ID

Fixtures Slice (`lib/store/fixtures-slice.ts`)

Action | Description

|---|---|

`setFixtures(fixture)` | Bulk replace fixtures (auto-sorts)

`generateFixtures(teams)` | Round-robin fixture generation

`addFixture(input, teams)` | Add single match

`reorderMatch(round, matchId, targetId)` | Drag-reorder within round

`updateMatch(id, field, value)` | Update single match field

`repairFixtures()` | Repair fixture locks/balances

Players Slice (`lib/store/players-slice.ts`)

Action | Description

|---|---|

`setPlayers(players)` | Bulk replace players

`addPlayer(player)` | Add single player

`updatePlayer(id, data)` | Partial update

`deletePlayer(id)` | Remove by ID

`deleteTeamPlayers(teamId)` | Remove all players of a team

`deleteAllPlayers()` | Clear all players

`importPlayers(csvText, teams)` | CSV import

`getTeamPlayers(teamId)` | Filter by team

``getAllPlayers()`` | Return all
``recalculateRatings()`` | Recompute all ratings

Persistence Behavior

- Only ``teams``, ``fixtures``, and ``players`` are persisted to `localStorage`
- On rehydration, orphaned matches (with deleted team IDs) are automatically cleared
- Auth state resets on page refresh (re-fetched from session endpoint)
- ``teamDataLoaded`` resets to ``false`` on login to trigger data refresh

10. Key Libraries & Utilities

Security (``lib/security.ts``)

Function	Purpose
``getClientIp(request)`` | Extract client IP from headers
``rateLimit({ key, limit, windowMs })`` | In-memory rate limiting (Map-based, resets on server restart)
``rateLimitResponse(resetAt)`` | 429 rate limit response with Retry-After header
``json(data, init?)`` | JSON response helper with security headers
``logSecurityEvent(event, details)`` | Security event logging (console.warn structured JSON)
``logApiError(event, error, details)`` | API error logging (console.error structured JSON)
``getAuthContext(supabase)`` | Extract auth context (admin/team/player) from session
``ownsTeam(auth, teamId)`` | Check if auth context owns a team
``requireAuth(auth)`` | Return 401 if not authenticated
``requireAdmin(auth)`` | Return 401/403 if not admin
``parseJsonObject(request)`` | Parse and validate JSON request body
``asString(value, maxLength?)`` | Validate string input
``asOptionalString(value, maxLength?)`` | Optional string input
``asInteger(value, min?, max?)`` | Validate integer input
``asBoolean(value)`` | Validate boolean input
``isValidEmail(email)`` | Email format validation
``validatePassword(password)`` | Password strength validation (12+ chars, upper+lower+number)
``sanitizeText(value)`` | Strip control characters and angle brackets

Player Credentials (``lib/player-credentials.ts``)

Function	Purpose
``normalizeCredentialPart(value)`` | Normalize to uppercase alphanumeric
``buildPlayerCredentialPair(name, team, seq)`` | Build username/password pair
``playerEmailFromUsername(username)`` | Build email from username
``isValidPlayerUsername(username)`` | Validate username format
``generatePlayerCredentialsForScope(options)`` | Full credential generation flow

Supabase Clients (``lib/supabase/``)

File	Client	Purpose
``client.ts`` | Browser client | Client-side Supabase operations
``server.ts`` | Server client (cookies) | Server-side authenticated operations
``service-role.ts`` | Service role client | Admin operations (bypasses RLS)
``public.ts`` | Anon key client | Public data access (live scores)
``middleware.ts`` | Middleware client | Session validation in middleware

Data Fetching Hooks (``lib/hooks/``)

Hook	File	Purpose
``refreshTeamData()`` | ``use-team-data.ts`` | Fetch team data from ``/api/team/data``
``refreshAdminData()`` | ``use-team-data.ts`` | Fetch admin data from ``/api/admin/data``
``usePlayerData()`` | ``use-player-data.ts`` | React hook for player dashboard data from ``/api/player/data``

Business Logic (``lib/logic/``)

Module	Purpose
``round-robin.ts`` | Round-robin fixture schedule generation

`standings.ts` | Standings computation from match results
`repair.ts` | Fixture repair (balancing home/away, resolving locks)

Utility Functions (`lib/utls/`)

Module	Purpose
`helpers.ts` | `sortMatchesByDateTime()`, ID generation, formatting
`csv.ts` | CSV parsing for player import
`data-import.ts` | JSON import parser with ID remapping

Generated on 2026-05-28 for the VUNA Football League Management System